

```

import random

from pokemon import create_default
from person import Person
from pokemon import Pokemon
from data_handling import table_print

# Battle Class
class Battle:
    __all_battles_list = []
    __total_battles = 0
    # Create two class attributes
    # 1. A list to track all battles
    # 2. An integer of the amount of battles

    # A static method to print all battles
    @staticmethod
    def print_all_battles():
        # Check to see if there are no battles
        if len(Battle.__all_battles_list) == 0:
            print("No battles to present.")
            return 0

        battle_list = [] # Create a list of the data from each battle:

        for battle in Battle.__all_battles_list: # The battle number, and the
name, owner, and kind of the two pokemon

            pokemon1 = battle.get_combatant(0)
            pokemon2 = battle.get_combatant(1)
            battle_list.append((battle.get_battle_num(), pokemon1.get_name(),
pokemon1.get_owner(),
                                pokemon1.get_kind(), pokemon2.get_name(),
pokemon2.get_owner(), pokemon2.get_kind(),
                                battle.get_result()))
            # Table print it (headers, data, widths)
            table_print(("Battle Number", "Pokemon 1 Name", "Pokemon 1 Owner", "Pokemon
1 Kind",
                        "Pokemon 2 Name", "Pokemon 2 Owner", "Pokemon 2 Kind",
"Result"),
                        battle_list, (15, 15, 15, 15, 15, 15, 15, 15))

        return 1

    # Initialize a battle object that takes in 2 pokemon objects
    def __init__(self, pokemon1, pokemon2):
        # Track both pokemon in a list
        self.__pokemon_list = [pokemon1, pokemon2]
        self.__result = "Battle has not happened yet"
        self.__battle_number = Battle.__total_battles

        Battle.__total_battles += 1
        Battle.__all_battles_list.append(self)
        # Create a variable that holds the result of the battle (initially set to
say battle has not happened yet)

        # Create a variable to get battle number (using class attribute for total
amount of battles)

```

```

# Update the class attribute integer and list

# Create to-string method (use this when you do print(battle_object))
# Includes the battle's number, names, and result
def __str__(self):
    if self.__result == "Battle has not happened yet":
        return f"Battle Number: {self.__battle_number}, Pokemon 1:
{self.__pokemon_list[0].get_name()}, " \
        f"Pokemon 2: {self.__pokemon_list[1].get_name()}, Result:
{self.__result}"

    return f"Battle Number: {self.__battle_number}, Pokemon 1:
{self.__pokemon_list[0].get_name()}, " \
        f"Pokemon 2: {self.__pokemon_list[1].get_name()}, Result:
{self.__result}"

# Starts the battle
def start_battle(self):
    # Create local variables for the two combatants
    pokemon0 = self.__pokemon_list[0]
    pokemon1 = self.__pokemon_list[1]

    # Let the user know that two Pokemon have appeared and their names
    print(f"Two Pokemon have appeared: {pokemon0.get_name()} and
{pokemon1.get_name()}")

    # If both Pokemon are owned, say the name of each Pokemon's owner and let
the user know that the Pokemon are battling.
    if pokemon0.get_owner() != "None" and pokemon1.get_owner() != "None":
        print(f"{pokemon0.get_owner()} 's {pokemon0.get_name()} is battling
{pokemon1.get_owner()} 's {pokemon1.get_name()}!")

    # If only one Pokemon is owned, let the user know that the owned Pokemon
was attacked by a 'wild' Pokemon.
    elif pokemon0.get_owner() != "None":
        print(f"{pokemon0.get_owner()} 's {pokemon0.get_name()} is being
attacked by a wild {pokemon1.get_name()}!")

    elif pokemon1.get_owner() != "None":
        print(f"{pokemon1.get_owner()} 's {pokemon1.get_name()} is being
attacked by a wild {pokemon0.get_name()}!")

    # If neither Pokemon is owned, say two 'wild' Pokemon are battling
    else:
        print(f"Two wild Pokemon, {pokemon0.get_name()} and
{pokemon1.get_name()}, are battling!")

    # Start the battle with round 0
    round_number = 0

    # Have the Pokemon battle!
    # This continues as long as both Pokemon have more than 0 HP
    while pokemon0.get_hp() > 0 and pokemon1.get_hp() > 0:
        # Have each Pokemon randomly choose a value between 1-3 to represent
their attack
        pokemon0_attack = random.randint(1, 3)
        pokemon1_attack = random.randint(1, 3)

```

```

        # Show the Round number and the name, kind, and HP of each Pokemon
        print(f"\nRound {round_number}:")
        print(f"{pokemon0.get_name()} ({pokemon0.get_kind()}) - HP:
{pokemon0.get_hp()}")
        print(f"{pokemon1.get_name()} ({pokemon1.get_kind()}) - HP:
{pokemon1.get_hp()}")

        # Pokemon 0 attacks first
        pokemon1.set_hp(pokemon1.get_hp() - pokemon0_attack)
        print(f"{pokemon0.get_name()} attacks {pokemon1.get_name()} for
{pokemon0_attack} damage!")

        # Output status of Pokemon 1
        print(f"{pokemon1.get_name()} now has {pokemon1.get_hp()} HP.")

        # Pokemon 1 attacks next
        pokemon0.set_hp(pokemon0.get_hp() - pokemon1_attack)
        print(f"{pokemon1.get_name()} attacks {pokemon0.get_name()} for
{pokemon1_attack} damage!")

        # Output status of Pokemon 0
        print(f"{pokemon0.get_name()} now has {pokemon0.get_hp()} HP.")

        # This round is now over, return to the top of the loop
        round_number += 1

    # The loop is over - time to find out why!
    # We'll return the winning Pokemon or a list of both Pokemon if there is a
tie
    # We'll also heal both Pokemon back to their starting values

    if pokemon0.get_hp() <= 0 and pokemon1.get_hp() <= 0:
        # II. Set both Pokemon's hp to the original HP value
        pokemon0.set_hp(self.__pokemon_list[0].get_hp())
        pokemon1.set_hp(self.__pokemon_list[1].get_hp())
        self.update_result("It's a tie!")
        return [pokemon0, pokemon1] # III. Return a list of both Pokemon for a
tie

    elif pokemon0.get_hp() <= 0:
        #Set both Pokemon's hp to the original HP value
        pokemon0.set_hp(self.__pokemon_list[0].get_hp())
        pokemon1.set_hp(self.__pokemon_list[1].get_hp())
        winner = pokemon1
        self.update_result(f"{winner.get_name()} is the winner!")
        return winner # Return the winning Pokemon

    elif pokemon1.get_hp() <= 0:
        #Set both Pokemon's hp to the original HP value
        pokemon0.set_hp(self.__pokemon_list[0].get_hp())
        pokemon1.set_hp(self.__pokemon_list[1].get_hp())
        winner = pokemon0
        self.update_result(f"{winner.get_name()} is the winner!")
        return winner #Return the winning Pokemon

# Getters and Setters for the Battle class
# These will be very useful for print_all_battles and start_battle

# Update the pokemon battling

```

```

def update_pokemon(self, pokemon1, pokemon2):
    self.__pokemon_list = [pokemon1, pokemon2]

# Update result of battle
def update_result(self, result):
    self.__result = result

# Get battle result
def get_result(self):
    return self.__result

# Get battle number
def get_battle_num(self):
    return self.__battle_number

# Get one of the pokemon objects
def get_combatant(self, number):
    if 0 <= number < len(self.__pokemon_list):
        return self.__pokemon_list[number]
    else:
        return None

# This function is used to print all of the people in our system
def print_people_list(person_list):
    # Initialize local variables
    people_data = []
    number = 0

    # Check to see if there are no people
    if len(person_list) == 0:
        print("No people to list.")
        return

    # Add all the people to our list
    for person in person_list:
        people_data.append((number, person.get_name(), person.get_email()))
        number += 1

    # Table print the list
    table_print( ("Number", "Name", "Email"), people_data, (10,10,10) )

# DO NOT DELETE - creates starting pokemon/people
# and adds people to a list to keep track of people
person_list = create_default()

# Create a user menu
# Make sure you work through the user menu and complete all the missing code
if __name__ == "__main__":

    while True:
        # Output a user menu with 8 Menu options
        print("\nWelcome to the Pokemon Battle Program!")
        print("Your options are:")
        print("1 - Add a Pokemon\n2 - See all Pokemon\n3 - Add a Person\n4 - See
all People "
                "\n5 - Adopt a Pokemon\n6 - Create a Battle\n7 - See all Battles\n8 -
Quit")
        user_input = input("Choose an option: ")

```

```

# Add a pokemon
if user_input == "1":
    print("\nAdd a Pokemon\n")
    name = input("What is the pokemon's name? ")
    kind = input("What is the pokemon's kind? ")
    pokemon_type = input("What is the pokemon's type? ")
    # Add a pokemon with the above attributes
    new_pokemon = Pokemon(name, kind, pokemon_type)
    Pokemon.add_pokemon(new_pokemon)

# See all pokemon
elif user_input == "2":
    print("\nSee all Pokemon\n")
    Pokemon.print_all_pokemon()

# Add a person
elif user_input == "3":
    print("\nAdd a Person\n")

    # Get person's name/email
    name = input("Enter the person's name: ")
    email = input("Enter the person's email: ")

    # Create person object and add to person list
    new_person = Person(name, email)
    person_list.append(new_person)

    # Let the user know the person has been added
    print(f"{new_person.get_name()} has been added to the list of people!")

# See all people
elif user_input == "4":
    print("\nSee all People\n")
    print_people_list(person_list)
    # print all the people

# Adopt a pokemon
elif user_input == "5":
    print("\nAdopt a Pokemon\n")

    print_people_list(person_list)
    person_num = input("Enter the number of the Person adopting: ")

    if not person_num.isdigit() or not (0 <= int(person_num) <
len(person_list)):
        print("Invalid person number. Please enter a valid number.")
        continue

    selected_person = person_list[int(person_num)]
    print("Pokemon available for adoption:")
    Pokemon.print_non_owned_pokemon()

    pokemon_num = input("Enter the number of the Pokemon to adopt: ")

    if not pokemon_num.isdigit():
        print("Invalid Pokemon number. Please enter a valid number.")

```

```

        continue

    # Check to see if the number is valid.
    non_owned_pokemon = Pokemon.print_non_owned_pokemon()
    if not (0 <= int(pokemon_num) < len(non_owned_pokemon)):
        print("Invalid Pokemon number. Please enter a valid number.")
        continue

    selected_pokemon = non_owned_pokemon[int(pokemon_num)]
    print(f"Selected Pokemon: {selected_pokemon.get_name()}, Owner:
{selected_pokemon.get_owner()}")

    if selected_pokemon.get_owner() is None:
        selected_pokemon.adopt(selected_person)
        print(f"{selected_person.get_name()} has adopted
{selected_pokemon.get_name()}!")
    else:
        print("Selected Pokemon is already adopted. Please choose another
Pokemon.")
        continue
# Create a battle (list all pokemon, ask them to choose 2 pokemon battling)
elif user_input == "6":
    print("\nCreate a Battle\n")

    # List all Pokemon
    Pokemon.print_all_pokemon()

    # Ask the user to choose the first Pokemon
    pokemon0_name = input("Enter the name of the first Pokemon: ")
    pokemon0 = Pokemon.find_pokemon(pokemon0_name)

    # Ask the user to choose the second Pokemon
    pokemon1_name = input("Enter the name of the second Pokemon: ")
    pokemon1 = Pokemon.find_pokemon(pokemon1_name)

    # Check if both Pokemon are valid
    if not pokemon0 or not pokemon1:
        print("Invalid Pokemon names. Please enter valid names.")
        continue # Return to the main menu

    # Create Battle object
    battle = Battle(pokemon0, pokemon1)
    print("Battle created!")

    # When the user hits Enter, start the battle
    input("Press Enter to Start the Battle!")
    battle.start_battle()

# See all battles, print out battle data
elif user_input == "7":
    print("\nSee all Battles\n")
    Battle.print_all_battles()

# End Program
elif user_input == "8":
    print("\nGoodbye!\n")
    break

# Error handling for bad input

```

```
elif user_input != "":  
    print("\nPlease enter a valid menu option\n")
```